ГУАП

КАФЕДРА № 41

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

Ассистент		Д.С. Лисюков
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ

Знакомство с функционалом сервиса Apache Airflow

по курсу: Облачные технологии и сервисы

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. № 4417		М. П. Курбатов
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Получить практические навыки развёртывания Apache Airflow 2.9.2 на персональном компьютере с помощью Docker Compose и ознакомиться с назначением основных компонентов сервиса.

2 Основные сведения

Apache Airflow используется для разработки, планирования и мониторинга рабочих процессов. DAG представляет собой направленный ациклический граф задач и их зависимостей. Оператор задаёт действие задачи, scheduler определяет готовность задач к запуску, а executor организует их выполнение. CeleryExecutor использует очередь сообщений и отдельные процессы worker; Celery не является обязательной основой любого планировщика Airflow.

В рассматриваемой конфигурации PostgreSQL хранит метаданные Airflow, Redis обслуживает очередь Celery, webserver предоставляет веб-интерфейс, scheduler планирует задачи, worker выполняет задачи, triggerer обслуживает отложенные задачи. Сервис airflow-init выполняет первоначальную подготовку базы и пользователя. Docker Compose описывает сервисы, их зависимости, переменные среды и подключённые каталоги.

3 Конфигурация проекта

Развёрнут Apache Airflow 2.9.2. В проекте созданы каталоги dags, logs, plugins, config и data и файл .env с AIRFLOW_UID=50000. Исходный файл сохранён как docker-compose.original.yaml, а для запуска подготовлена рабочая копия docker-compose.yaml. Каталог data подключён к /opt/airflow/data. Загрузка примеров отключена параметром AIRFLOW__CORE__LOAD_EXAMPLES=false. Для локального доступа к PostgreSQL опубликован порт 127.0.0.1:5433:5432. На рисунке 1 показан промежуточный этап до создания рабочей копии Compose; последующий запуск подтверждён рисунками 3 и 4.

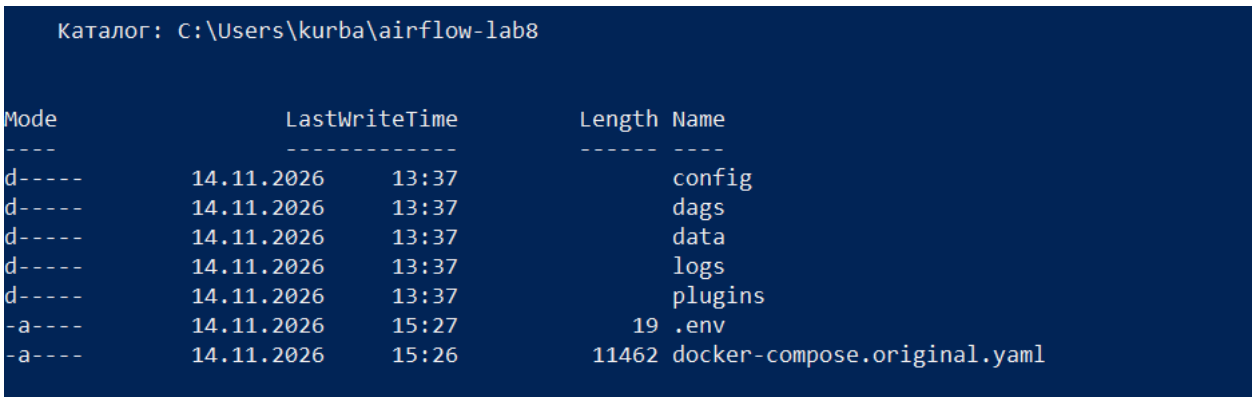


Рисунок 1 — Каталоги проекта, .env и исходная копия Compose

```
  # build: .
  environment:
    &airflow-common-env
    AIRFLOW__CORE__EXECUTOR: CeleryExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
    AIRFLOW__CELERY__RESULT_BACKEND: db+postgresql://airflow:airflow@postgres/airflow
    AIRFLOW__CELERY__BROKER_URL: redis://:@redis:6379/0
    AIRFLOW__CORE__FERNET_KEY: ''
    AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
    AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
    AIRFLOW__API__AUTH_BACKENDS: 'airflow.api.auth.backend.basic_auth,airflow.api.auth.backend.session'
    # yamllint disable rule:line-length
    # Use simple http server on scheduler for health checks
    # See https://airflow.apache.org/docs/apache-airflow/stable/administration-and-deployment/logging-monitoring/check-hea
    # yamllint enable rule:line-length
    AIRFLOW__SCHEDULER__ENABLE_HEALTH_CHECK: 'true'
    # WARNING: Use _PIP_ADDITIONAL_REQUIREMENTS option ONLY for a quick checks
    # for other purpose (development, test and especially production usage) build/extend Airflow image.
    _PIP_ADDITIONAL_REQUIREMENTS: ${_PIP_ADDITIONAL_REQUIREMENTS:-}
    # The following line can be used to set a custom config file, stored in the local config folder
    # If you want to use it, outcomment it and replace airflow.cfg with the name of your config file
    # AIRFLOW_CONFIG: '/opt/airflow/config/airflow.cfg'
    # AIRFLOW_CONFIG:  /opt/airflow/config/airflow.cfg
  volumes:
    - ${AIRFLOW_PROJ_DIR:-.}/dags:/opt/airflow/dags
    - ${AIRFLOW_PROJ_DIR:-.}/logs:/opt/airflow/logs
    - ${AIRFLOW_PROJ_DIR:-.}/config:/opt/airflow/config
    - ${AIRFLOW_PROJ_DIR:-.}/plugins:/opt/airflow/plugins
    - ${AIRFLOW_PROJ_DIR:-.}/data:/opt/airflow/data
  user: "${AIRFLOW_UID:-50000}:0"
  services:
    postgres:
      image: postgres:13
      environment:
        POSTGRES_USER: airflow
        POSTGRES_PASSWORD: airflow
        POSTGRES_DB: airflow
      volumes:
        - postgres-db-volume:/var/lib/postgresql/data
      ports:
        - "127.0.0.1:5433:5432"
      healthcheck:
        test: ["CMD", "pg_isready", "-U", "airflow"]
        interval: 10s
        retries: 5
        start_period: 5s
      restart: always
  flower:
    <<: *airflow-common
    command: celery flower
    profiles:
      - flower
    ports:
      - "5555:5555"
    healthcheck:
      test: ["CMD", "curl", "--fail", "http://localhost:5555/"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 30s
    restart:  unless-stopped
    depends_on:
      <<: *airflow-common-depends-on
      airflow-init:
        condition: service_completed_successfully

volumes:
  postgres-db-volume:
```

Рисунок 2 — Фрагменты Compose: отключение примеров, каталог data, порт PostgreSQL и политика перезапуска Flower

4 Инициализация и запуск сервисов

Первоначальная подготовка выполнена командой docker compose up airflow-init. Во время инициализации созданы таблицы метаданных и пользователь интерфейса. В списке сервисов airflow-init имеет состояние Exited (0), что подтверждает успешное завершение. Основные сервисы запущены командой docker compose up -d. Scheduler, worker, triggerer, webserver, PostgreSQL и Redis в представленном выводе имеют статус healthy. Остановленный airflow-init является штатным состоянием одноразового сервиса.

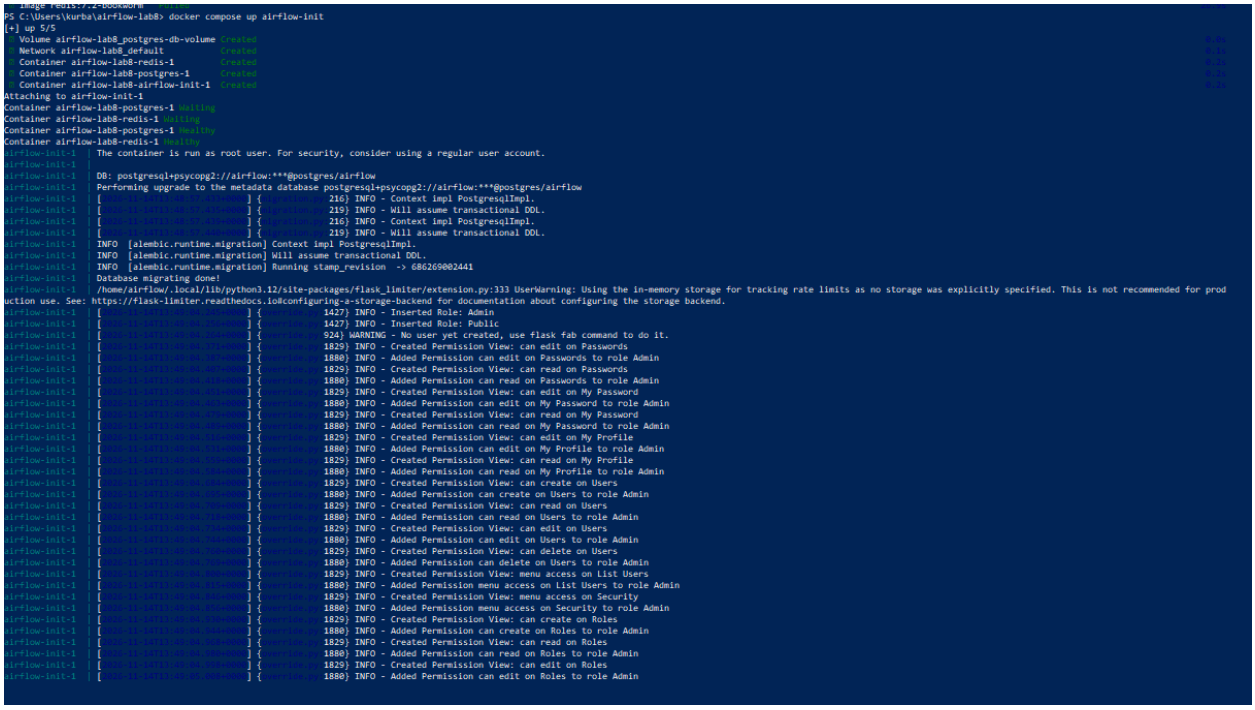


Рисунок 3 — Инициализация базы метаданных Airflow

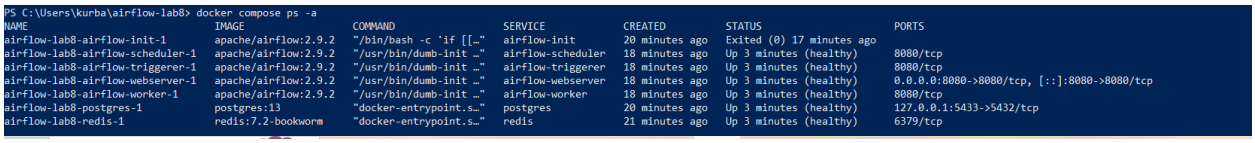


Рисунок 4 — Состояние сервисов Airflow после запуска

5 Доступ к веб-интерфейсу

После изменения внешнего порта веб-интерфейс открыт по адресу http://127.0.0.1:8081. Вход выполнен под учебной учётной записью airflow. После авторизации открылась страница DAGs, на которой показано 0 DAG. Это соответствует отключённой загрузке примеров и отсутствию собственных DAG в каталоге проекта. Создание и запуск собственного рабочего процесса в

данном задании не требуются. Снимок сервисов на рисунке 4 сделан до смены порта, поэтому на нём ещё указан 8080; итоговый адрес виден на рисунке 5.

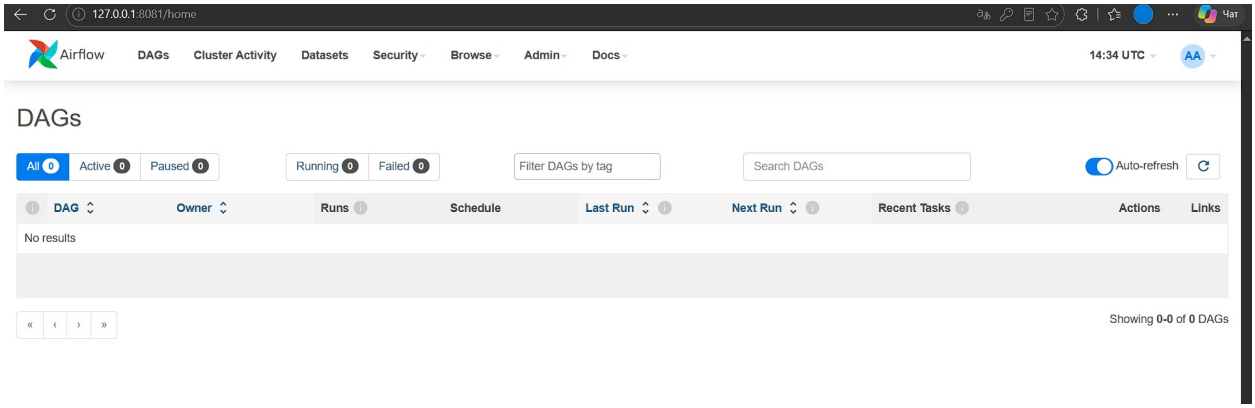


Рисунок 5 — Веб-интерфейс Airflow после входа: список DAG без демонстрационных примеров

6 Трудности и результат проверки

При скачивании Compose через curl.exe возникла ошибка CRYPT_E_REVOCATION_OFFLINE: Windows не смогла проверить отзыв сертификата. Файл был получен через браузер и сохранён как docker-compose.original.yaml. Команда docker compose config --quiet первоначально сообщала no configuration file provided, поскольку Compose не находил файл со стандартным именем. Для запуска создана рабочая копия docker-compose.yaml.

При загрузке образов Redis и Airflow неоднократно появлялся TLS handshake timeout при обращении к auth.docker.io. Загрузка проверялась отдельно командой docker pull apache/airflow:2.9.2 и через docker compose pull. Для ограничения одновременных операций использовалась переменная COMPOSE_PARALLEL_LIMIT=1. Повторные попытки в итоге завершились успешно: Airflow, PostgreSQL и Redis получили состояние Pulled. Единственная точная причина сетевых тайм-аутов по имеющимся данным не установлена.

Во время загрузки возникали input/output error и read-only file system при записи в хранилище Docker. В ходе диагностики проверялось свободное место на диске C:, обсуждалась очистка временных файлов и отключение гибернации. Проверка записи выполнялась созданием тома docker-write-check;

для восстановления предлагались освобождение места и перезапуск Docker/WSL. В последующей проверке операция с томом прошла, а затем все необходимые образы были загружены. Файлы виртуальных дисков и данные MySQL не удалялись в рамках предложенных действий.

При первом запуске основных сервисов контейнер PostgreSQL завершился с кодом 139. В логах позднее были видны восстановление после некорректного завершения и сообщение database system is ready to accept connections. Контроль docker compose ps -a подтвердил статус healthy у PostgreSQL и Exited (0) у airflow-init. Ранние сообщения relation does not exist рассматривались в контексте подготовки схемы; окончательный результат оценивался по состоянию сервисов, а не по отдельным старым строкам лога.

После запуска браузер сначала показывал страницу welcome-контейнера предыдущей работы, затем ERR_EMPTY_RESPONSE. В PowerShell запрос curl.exe --noproxy возвращал Empty reply from server. При этом проверка /login/ внутри airflow-webserver отвечала HTTP 200 OK, а /health также возвращал 200. Это позволило отделить работоспособность Airflow от доступа через сеть Windows. Внешний порт был изменён с 8080 на 127.0.0.1:8081:8080, веб-сервер пересоздан; дополнительно выполнялись проверки публикации порта и предлагался перезапуск Docker/WSL. Смена порта не дала немедленного результата, однако позже страница входа открылась, а после авторизации стал доступен интерфейс DAGs. По имеющимся данным нельзя приписать восстановление одному конкретному действию.

7 Вывод

Развёртывание Airflow демонстрирует, как Docker Compose объединяет несколько взаимосвязанных сервисов в одно приложение. Для его работы недостаточно открыть веб-страницу: необходимы база метаданных, очередь сообщений и процессы планирования и выполнения задач. Отдельная инициализация подготавливает инфраструктуру до запуска основных

компонентов, поэтому её успешное завершение проверяется отдельно от состояния постоянно работающих сервисов.

Подключение каталогов отделяет файлы проекта от жизненного цикла контейнеров, а публикация портов обеспечивает доступ к веб-интерфейсу и PostgreSQL с персонального компьютера. При диагностике важно различать конфликт порта, нехватку ресурсов, неуспешную миграцию базы и ошибки доступа к файлам. Практический результат развёртывания подтверждается кодом завершения airflow-init, состоянием сервисов и доступностью интерфейса после авторизации.

В результате Apache Airflow 2.9.2 развёрнут на персональном компьютере с помощью Docker Compose. Успешность подтверждена завершением airflow-init с кодом 0, здоровым состоянием основных сервисов и открытием страницы DAGs после авторизации по адресу 127.0.0.1:8081. Наиболее существенными трудностями стали доступ к реестру образов, ошибки записи хранилища Docker и доступ из Windows к опубликованному веб-интерфейсу. Раздельная проверка загрузки образов, состояния базы, ответа приложения внутри контейнера и доступа с хоста позволила последовательно определить этапы сбоя.

Список использованных источников

1. Знакомство с функционалом сервиса Apache Airflow : методические указания к практической работе. 12 с.

2. Apache Airflow 2.9.2. Running Airflow in Docker. URL: https://airflow.apache.org/docs/apache-airflow/2.9.2/howto/docker-compose/index.html.

3. Docker Compose. URL: https://docs.docker.com/compose/.